中央学生助手（CampusBill）技术文档

意图控制、架构约束与可复现性论证

项目	内容
作品	中央学生助手（CampusBill）——离线优先的校园账单 App + DeepSeek 对话助手
仓库	Student-Spending-Bill
分支	feat/assistant-competition
代码基线	HEAD 910041eda1e70afe846bd3492167025d7fe8f667（该提交只改演示脚本，代码树与 966f41c 相同）
文档版本	v1.0（2026-11-12）
对应赛题要求	硬性要求④：技术文档 PDF，不超过 30 页；重点为意图控制策略、架构设计与可复现性论证
生成方式	本册由 `docs/assistant/tech-doc/render.ps1` 从本 Markdown 生成 PDF，页数可自校验
阅读顺序	第 1 章看边界 → 第 2、3 章看设计与意图控制 → 第 4 章看约束 → 第 5 章照着跑 → 第 6 章核对证据 → 第 7 章看局限

> 本册只写能对上代码或产物的事实。凡是本轮没有实测的，正文明确写「未实测」，并在 §7.2 汇总，不用估计值冒充指标。

目 录

1 文档目的与事实基线 3
1.1 赛题四项硬性要求与本册的对应 3
1.2 一句话定位与三条设计红线 3
1.3 本册引用的事实基线 3
2 系统架构 4
2.1 分层与模块边界 4
2.2 一轮对话的时序（含确认挂起与恢复） 4
2.3 线程模型与状态机 5
2.4 账本数据模型与统计口径 5
2.5 文档即协议：docs/assistant 单一源 6
2.6 代码规模与模块分工 6
3 意图控制策略 7
3.1 三层路由：什么时候不交给模型 7
3.2 判定表：问句 → 意图 → 工具 → 兜底 7
3.3 查账参数推导：时间窗、月度口径与食堂专名 8
3.4 模型漏发工具时的补偿与工具优先级 8
3.5 应用侧数字闸门（ReplyGate） 9
3.6 提示词与上下文的组装顺序 9
3.7 同一句话的七种走向 9
4 工具、确认与安全约束 11
4.1 四个工具与 schema 约束 11
4.2 写路径确认门（从 draft 到回读一致） 11
4.3 密钥与隐私边界 12
4.4 过程留痕与脱敏 12
4.5 失败分类与四档状态的封闭性 12
4.6 参数边界值：坏输入怎么收场 13
5 可复现性论证 14
5.1 环境与前置条件 14
5.2 构建、安装与测试命令 14
5.3 演示数据与可核对数字 15
5.4 本文档自身的构建与页数自检 15
5.5 不联网可复现 / 需联网才能复现的边界 16
5.6 复现失败时的排查矩阵 16
6 证据索引 17
6.1 代码锚点总表 17
6.2 构建、测试与产物证据 18
6.3 协议文档清单 19
6.4 单元测试清单（26 个方法） 19
7 已知局限与风险 21
7.1 已知文档/代码不一致 21
7.2 未实测项（明确标注） 21
7.3 设计取舍的副作用 21
7.4 下一步可验证清单 22
附录 A 术语表 23
附录 B 评审速查（3 分钟） 24
附录 C 评审质疑与证据回应 25

1 文档目的与事实基线

1.1 赛题四项硬性要求与本册的对应

本赛题对交付物有四项硬性要求：可访问的代码仓库与提交历史、可复现的构建与运行方式、5–8 分钟演示视频、不超过 30 页的技术文档。前三项分别由仓库本身、`docs/assistant/REPRODUCE.md` 与演示脚本承担；本册承担第四项，并刻意做成「可核对」而不是「可阅读」：每一个关键论断后面都跟一条能在仓库里定位的证据（文件:行号、命令、或产物摘要）。

`docs/assistant/SUBMISSION-CHECKLIST.md:25-26` 把技术文档的评审重点写得比较具体：意图控制策略（上下文与 Prompt 设计）、架构设计（AI 生成代码的架构及如何约束）、可复现性论证。本册的章节就是按这三点展开的，另加证据索引与局限清单，避免评审只能靠信任。

要求	承担物	本册对应章节
① 代码仓库与提交历史	git 仓库（分支 feat/assistant-competition）	§1.3、§6.2
② 可复现构建与运行	`REPRODUCE.md`、Gradle wrapper、固定依赖	§5.1–§5.3
③ 5–8 分钟演示视频	`DEMO-SCRIPT.md`（分镜与话术）	§5.3（演示数字口径）
④ 技术文档 ≤30 页	本册（Markdown + PDF）	全册；页数自校验见 §5.4

1.2 一句话定位与三条设计红线

中央学生助手是一个「记账 App 为主、AI 对话为辅」的作品：账单数据、统计、导入、预算都在本机完成，DeepSeek 只负责自然语言这一层。这个定位不是宣传语，而是三条可以检查的红线：

1. **金额不能由模型生成。** 任何与账本有关的数字，必须来自本轮的 `query_spend` 结果或刚写入记录的回读；模型自己写出来的数字在应用侧被拦掉（§3.5，`ReplyGate.java:11-13`）。
2. **写库不能由模型直接完成。** 模型只能提出 `propose_add_bill` 建议，真正写入发生在用户点确认之后，且写入后必须回读比对（§4.2，`AssistantController.java:174-208`）。
3. **没有 Key 也必须能用。** 四个首页快捷句与旧版离线意图继续走本地 `ReplyGenerator`，不发网络请求；没有 Key 时对话入口给出确定的错误态，而不是假装在思考（§4.5，`AssistantController.java:138-141`）。

这三条红线决定了后文的架构：本地路由器先判、模型只做「语言外壳」、应用侧再做一次结果校验。换句话说，AI 被放在一个「可拒绝、可回读、可降级」的位置上，而不是被当成账本的唯一真相来源。

1.3 本册引用的事实基线

下表是本轮（2026-11-12 13:52 前后）亲自复算的基线；「来源」列写的是取得该数字的动作，不是文档转述。

事实	值	来源
代码基线	HEAD `b2a24376c96035940f8c2684419f139db26b6d36`（代码树同 `966f41c`；`910041e` 仅改 S6 算术，`b2a2437` 仅改 DEMO-SCRIPT 锚点与交叉引用）	`git rev-parse HEAD`；源码零改动
单元测试	8 个测试类、51 个测试，0 失败 / 0 错误 / 0 跳过；XML 时间戳 2026-11-12T05:52:11.497Z–05:52:11.634Z（本地 13:52:11）	解析 `app/build/test-results/testDebugUnitTest/*.xml`
Debug APK	`app/build/outputs/apk/debug/app-debug.apk`，26,702,585 字节	`Get-Item` 长度；`Get-FileHash`
APK SHA-256	`B5A4B94991897FCCFB08C6FCF652F5FC0651615909DACE4C9AAD053E29B6CFC6`	同上
代码规模	助手包 30 个 Java 文件 / 2,391 行；全工程 92 个文件 / 11,707 行	脚本统计（§6.2 可复跑）
构建与数据库	compileSdk 37 / minSdk 29 / targetSdk 36；`com.campusbill.app` v1.0；Room `student_bill.db` v3（两条迁移）	`app/build.gradle:7,10-14`、`AppDatabase.java:12-14`

表里只放「能被第三方用一条命令重新得到」的量；需要仪器与多次采样的指标（首屏耗时、端到端延迟）一律不写，理由见 §7.2。

2 系统架构

2.1 分层与模块边界

工程按「界面 / 编排 / 工具 / 数据 / 协议文档」五层切开。分层的判据不是目录好看，而是「换掉任意一层，其余层不需要改代码」：协议文档换成新版本，只影响 `docs/assistant` 与 assets 同步；模型从 DeepSeek 换成别的兼容接口，只影响 `DeepSeekStreamClient`；账本从 Room 换成别的存储，只影响 `RoomBillLedger`。

层	关键类型	职责	不做什么
界面	`AiChatActivity`（1,314 行）、`AssistantHost`（153 行）	输入、气泡、四个快捷芯片、确认卡、状态文案、主线程回调	不拼工具参数、不判定金额是否可信
编排	`AssistantController`（536 行）、`AssistantStatusMachine`	一轮对话的轮次控制、工具调用、确认挂起/恢复、失败归档	不直接读 SQLite、不直接读写资产文件
工具	`SpendQuery`（220 行）、`FortuneComposer`、`DiaryNoteStore`、`BillDraft`	纯函数式查询/组装/解析，输入账本列表，输出 JSON	不弹 UI、不持有 Context
数据	`RoomBillLedger`（67 行）、`BillContextBuilder`（189 行）、`AppDatabase`	账本读写、上下文摘要（Top5/最近 15 条）	不生成自然语言
协议	`AssistantDocuments`、`SystemPromptBody`、`ToolSchemaCatalog`	从 assets 读 4 份协议文档，抽取正文/schema/版本	不在代码里另写一份 schema

界面层里有一个容易误读的细节：`AssistantHost` 不是「AI 层」，它只是 `AssistantController` 的生命周期壳。它把 `main` Handler、单线程 `Executor`、以及「现在有没有 Key」注入控制器（`AssistantHost.java:46-78`），并把所有回调 post 回主线程（`AssistantHost.java:61,66,71,76`）。这样控制器可以在后台线程里跑 SSE，而 UI 永远只在主线程被碰。

2.2 一轮对话的时序（含确认挂起与恢复）

下面这张时序表把「用户说一句话」到「屏幕上出现一行字」拆成 11 步。每一步都给出代码位置，评审可以照着跳读。注意第 6 步的「挂起」：确认类工具不会继续跑模型，而是把这一轮停在 `AWAITING_CONFIRM`，等用户点按钮后才由第 7 步续上。

步	动作	代码位置	状态	用户可见
1	判断这句话要不要交给助手	`AiChatActivity.java:283-296`	空闲	输入框清空、插入用户气泡
2	抢占会话（防并发）	`AssistantHost.java:94-107`	空闲→思考	状态行「思考」
3	记录用户轮、算本轮是否需要查账	`AssistantController.java:132-137`	THINKING	—
4	拼 system（含上一会话要点）+ 发 SSE	`AssistantController.java:460-469`、`:252`	THINKING	思考摘要只进日志
5	流式正文过数字闸门后上屏	`AssistantController.java:259-266`	THINKING/TOOL	逐字出现
6	有工具调用则按优先级执行；确认类工具挂起	`AssistantController.java:301-343`、`:345-374`	TOOL→等待确认	确认卡/查询结果
7	用户点确认：写入→回读→比对→回执	`AssistantController.java:174-208`	等待确认→THINKING	「已写入并回读：…」
8	用户点取消：不写库，给固定话术	`AssistantController.java:221-241`	等待确认→空闲	「…我还没有写入。」
9	本轮工具跑完，进入下一模型轮	`AssistantController.java:342`、`:243-247`	THINKING	—
10	达到轮次上限或正文结束，收状态	`AssistantController.java:243-247`、`:292-295`	空闲	球回 idle
11	失败则归档为 ERROR 并收起确认卡	`AssistantController.java:393-402`	ERROR	一行错误原因

第 6 步的「挂起」是这套设计里最值得解释的一处。普通的工具调用（查账、运势）可以在同一轮里跑完再回模型；但「记账」和「引用日记」会改状态，所以 `beginConfirm` 返回 `true` 表示「本轮到此为止，等确认」

（`AssistantController.java:310-315`）。这样做的副作用是：模型不能在用户点确认之前「继续说」，也就不可能发生「AI 先说已记好、用户还没点」的错位。测试 `demo_确认前不写库确认后回读一致` 就在守这条边界（`AssistantCoreTest.java:28-472`）。

2.3 线程模型与状态机

线程模型只有三条规则，全部可以在 `AssistantHost.java` 里对上：

1. **所有 UI 回调都在主线程。** 控制器不直接碰 View，只调用 `Ui` 接口；`AssistantHost` 的实现把每个回调 `main.post(...)`（`AssistantHost.java:61,66,71,76`）。
2. **一轮对话串行。** `startOrIgnore` 用 `busy.compareAndSet(false, true)` 抢占（`AssistantHost.java:94-107`）；抢不到就 post 一个 `onIgnored` 并返回 false。`confirm` 与 `cancel` 复用同一个单线程 `executor`（`AssistantHost.java:123-129`），所以「确认」不会和「新一轮」并发。
3. **SSE 在后台线程阻塞读。** `DeepSeekStreamClient` 的类注释明确要求调用方在后台线程使用（`DeepSeekStreamClient.java:28`），连接/读取超时设为 10s/60s（`:59-60`）。

状态机只有四档，且「空闲」不是第五档，而是 `machine.clear()` 后的空字符串（`AssistantStatus.java:9-12`，`AssistantStatusMachine.java:24-26`）。这个封闭性是有意的：界面文案、球映射、确认卡的显隐都只依赖这四个值，任何新状态都必须先改枚举，评审可以一眼看出有没有偷偷加状态。

状态	文案	球的默认 id	界面含义
THINKING	思考	30	已发请求，等首个 token
TOOL	调用工具	32	正在查账/组装结果
AWAITING_CONFIRM	等待确认	03	有确认卡，输入被挂起
ERROR	出错	34	本轮终止，可重试

球的 id 不是硬编码在界面里，而是先读 `emotion-map.v0.1.md` 的 `hook:` 行，读不到才用默认值（`EmotionHooks.java:26-30`）。场景映射另有一套（`EmotionHooks.java:80-109`）：例如查账工具用 40、普通流式用 39、等待确认固定 03 而「不复用 35」。这套双轨（状态档位 + 场景）让界面既能表达「在干嘛」，也能表达「情绪如何」，而不必给状态机加第五档。

2.4 账本数据模型与统计口径

账本只有一本。`RoomBillLedger.all()` 直接调 `dao.getAllBills()`，注释写得很直白：「没有 batch_id 条件，历史导入和手动记下的行都在这本账里」（`RoomBillLedger.java:22-29`）。这是刻意的产品决定：助手回答「我一共花了多少」时，如果漏掉导入的历史账单，答案就是错的。测试 `querySpend_all含导入与手动且合计不因明细截断` 和 `模型把历史问成这周时仍查全部账本` 守的就是这一点。

写入路径也只有一个：`insertAndRead` 先 `insert`，再把 `batchId` 置 0，最后用自增 id `getById` 回读（`RoomBillLedger.java:37-50`）。回读不是形式主义——它是确认卡「回执一致」的唯一证据来源（§4.2）。

统计口径有三处容易出错，代码里都做了显式处理：

- **金额按分累加。** `SpendQuery` 用 `long cents += Math.round(amount * 100.0)`，最后 `cents / 100.0`（`SpendQuery.java:101-105`）。这样 0.1+0.2 不会变成 0.30000000000000004。
- **只统计支出。** `typeMatches` 把 `income` 映射到 1、`expense` 映射到 0（`SpendQuery.java:161-169`），查询默认 `type=expense`。
- **明细截断但合计不截断。** 返回的 `items` 最多 20 条（`SpendQuery.java:128-140`），但 `total`/`count` 基于全部命中行。这是「回答金额要准、上下文要省」的折中，副作用写在 §7.3。

「当月」的口径需要单独说明：它指首页折线图当前选中的月份，不是自然月。`SpendQuery` 在 `period=chart_month|this_month` 时用传入的 `chartMonth` 整月（`SpendQuery.java:68-70`），而 `chartMonth` 由界面传入（`AssistantHost.java:82-84`）。`SYSTEM_PROMPT.v0.1.md:23` 把这条写成硬规则，测试 `当月跟随首页选中月_点名月份用那个月` 与 `querySpend_thisMonth按选中月而不是本周` 各守一半。

2.5 文档即协议：docs/assistant 单一源

这个项目有四份「协议文档」：`SYSTEM_PROMPT.v0.1.md`、`tools.schema.v0.1.json`、`intent-control.v0.1.md`、`think-protocol.v0.1.md`，外加情绪映射 `emotion-map.v0.1.md`。它们不是说明文档，而是运行时输入：App 启动时从 assets 读出来，system prompt 用正文、工具表用 schema、球映射用 hook 行。

单一源由构建保证：`app/build.gradle:34-42` 注册了 `syncAssistantDocs` 任务，把 `${rootDir}/docs/assistant` 整个复制到 `src/main/assets/assistant`，而 `assemble` 依赖它。也就是说，仓库里只有 docs 一份源；assets 里的同名文件是构建产物。`AssistantDocuments.java:10` 的注释把这条写成约束：「文件名与 DSH v0.1 一致，这里不另写一套 schema」。

协议版本也有痕迹：每轮开始时会记一条 `prompt` 事件，字段是 `version` 与 system prompt 正文的 SHA-256（`AssistantController.java:135-136`）。这意味着「这轮到底用了哪版提示词」不靠人记，而是可以从过程日志里回查（§4.4）。如果评审想验证「文档改了但代码没改」，看这条事件的 hash 变化即可。

2.6 代码规模与模块分工

助手功能没有散落在 Activity 的 if-else 里，而是集中在 `com.campusbill.app.chat.ai.assistant` 包内：本轮统计 30 个文件、2,391 行。按职责分七组，评审可以按组抽读：

分组	代表文件	职责	关键约束
编排	`AssistantController`、`AssistantHost`、`AssistantStatusMachine`	轮次推进、确认挂起、主线程回调	最多 4 轮（`AssistantController.java:55`）
意图	`AssistantIntentPlanner`、`ReplyGate`	本地路由与数字闸门	命中词表才查账，带数字回复先拦（`ReplyGate.java:11-13`）
查询	`SpendQuery`、`RoomBillLedger`	时间窗、按分累加、汇总 JSON	一本账，无 `batch_id` 过滤（`RoomBillLedger.java:22-29`）
记账	`BillDraft`、`RoomBillLedger`	参数兜底、校验、回读	金额不兜底，缺失就回问（`BillDraft.java:60-62`）
协议	`AssistantDocuments`、`SystemPromptBody`、`ToolSchemaCatalog`、`EmotionHooks`	docs → assets → 运行时	单一源，构建期同步（`app/build.gradle:34-42`）
网络	`DeepSeekStreamClient`、`FailureClassifier`、`AiConfig`	SSE 解析、三类失败	连接 10s / 读取 60s（`DeepSeekStreamClient.java:59-60`）
安全与留痕	`DeepSeekKeyStore`、`SecretRedactor`、`ProcessEvents`、`FileProcessLog`	Key 加密、日志脱敏	Key 不进日志（`SecretRedactor.java:14-15`）

这张表也解释了「AI 生成代码怎么约束」：每组只做一件事，跨组只通过接口（`Ledger`、`Streamer`、`Ui`）通信；控制器不知道 Room 的表结构，`SpendQuery` 不知道网络的存在。约束不靠评审时的口头约定，而靠「这一组没有权限做那件事」的依赖方向。

3 意图控制策略

3.1 三层路由：什么时候不交给模型

一句话进 App 后，先过三层判断，只有第三层才会碰到网络。这个顺序不是随意排的：越靠前的层越便宜、越确定、越离线可用。

层	判据	结果	代码
第一层：本地快捷句	文本命中 `本月总支出` / `预算` / `省钱建议` / `消费分类`，或问「怎么导入/怎么记账」	直接走本地 `ReplyGenerator`，不发请求	`AssistantIntentPlanner.java:116-124`、`AiChatActivity.java:306-422`
第二层：助手计划	`AssistantIntentPlanner.plan(...)` 返回非空	交给 `AssistantController`（可能联网）	`AiChatActivity.java:283-284`
第三层：兜底	第一层不命中、计划为空、且有 Key	`replyOnlineOrFallback`：带 `BillContextBuilder` 摘要请模型自由回答，失败再降级	`AiChatActivity.java:424-452`

界面上的判断只有两行，但两行都重要：

```
boolean assistantUtterance =
        !AssistantIntentPlanner.plan(query, LocalDate.now(),
                YearMonth.parse(queryMonth)).isEmpty();
boolean streamedChat =
        IntentMatcher.match(query) == IntentMatcher.Intent.FALLBACK
        && AiConfig.hasApiKey(this);
```

（`AiChatActivity.java:283-286`，为排版折行，语义未改。）

第一行决定「这句话是不是助手该管的」；第二行决定「这句话是不是旧意图没覆盖、且有 Key」。两者任一为真就走 `assistantHost.startOrIgnore`（`:287-296`）；否则本地占位 + `dispatch`（`:297-303`）。这个设计带来一个可预测的行为：**没有 Key 时，助手类问句仍会进入控制器，然后以 `API_KEY` 失败态收场**（`AssistantController.java:138-141`），而不是悄悄退回本地答一个不相干的回复。错误是显式的，评审在无 Key 机器上也能复现（§5.5）。

3.2 判定表：问句 → 意图 → 工具 → 兜底

这张表是本册的核心。每一行都能在 `AssistantIntentPlanner` 或 `AiChatActivity` 里找到对应分支，且都有测试或可复现路径。

用户原话（示例）	命中路径	意图/工具	关键参数	兜底与可见结果
本月总支出（点芯片）	第一层	本地 `ReplyGenerator.totalMonth`	无	不发网络；直接出本月合计
预算剩余（点芯片）	第一层	本地 `ReplyGenerator.budget`	无	不发网络；出预算/剩余
这周在食堂花了多少	第二层	`query_spend`	`period=this_week`，`merchant_contains=食堂`，`category=食堂`	查到才显示数字；失败「这次没有查成账本。」
这个月花了多少	第二层	`query_spend`	`period=chart_month`（跟随首页选中月）	同上
3 月花了多少	第二层	`query_spend`	`period=custom`，`start=2026-03-01`，`end=2026-03-31`	裸月用选中月年份
一共花了多少 / 导入的账单里花了多少	第二层	`query_spend`	`period=all`，清空起止日期	含导入与手动，合计不截断

用户原话（示例）	命中路径	意图/工具	关键参数	兜底与可见结果
记一笔 18 元午餐	第二层	`propose_add_bill`	`amount=18`，`category=餐饮`，`date=今天`	弹确认卡；确认后写入并回读
记一笔（没说金额）	第二层	`propose_add_bill`	无 `amount` → `valid()` 失败	不弹卡，回「还缺金额，我没有写入。」
今天运势怎么样	第二层	`daily_fortune`	`style=fortune`（塔罗→`tarot_lite`，晴雨→`mood`）	不弹确认卡；直接出文案
用我的日记跟我打个招呼	第二层	`request_diary_context`	`purpose=greeting`	弹确认卡；确认前不引用原文
怎么导入账单	第一层	本地 `HELP`	无	不查账、不联网
你好 / 在吗	第三层（有 Key）	无工具，自由对话	注入账本摘要	无 Key 时走本地 `fallback`

这张表里有两处「反直觉但刻意」：其一，「预算」是子串命中，所以任何含「预算」的句子都会留在本地（§7.3 记录其副作用）；其二，「运势」在 `isQuery` 里被排除，但只在「同时不提钱、不提账」时排除（`AssistantIntentPlanner.java:93-95`），所以「这周运势和花销」仍然会先查账——优先级是查账 > 运势。

3.3 查账参数推导：时间窗、月度口径与食堂专名

工具参数不是模型自由发挥，而是由 `queryArgs` 从原句推导（`AssistantIntentPlanner.java:208-231`），模型只负责「说人话」。推导规则按优先级排列：

原句特征	`period`	附加参数	理由
含「这周/本周/这个星期/本星期」	`this_week`	—	与首页周视图一致，周一到周日（`WeekRange`）
含「今天/今日」，且没有选中月词、没有点名月	`today`	—	单日查询
含「2026年3月」或裸「3月」	`custom`	`start_date` / `end_date` 为整月	裸月用选中月年份（`:159-177`）
含「当月/本月/这个月」	`chart_month`	—	跟随折线图选中月，不是自然月
其它（含「一共/总共/历史/习惯/建议」）	`all`	清空起止	见 §3.4 的「全部账本」规则

除时间窗外，还有两条固定规则：`type` 一律 `expense`（`:225`）；原句含「食堂」时同时设 `merchant_contains=食堂` 与 `category=食堂`（`:226-229`）。后者是为数据现状做的兜底：导入账单的商户名可能是「第一食堂」「第二食堂」，分类也可能被写成「餐饮」，两个条件同时给，`textMatches` 在「商户词等于分类词」时改走或匹配（`SpendQuery.java:171-179`），从而两边都能命中。

`period=all` 还有一条应用层修正：即便模型给了具体区间，只要原句没有显式时间窗，`spendArgs` 会把 `period` 改成 `all` 并删掉 `start_date` / `end_date`（`AssistantController.java:441-450`）。这条修正防的是「模型自作主张把历史问题答成本周」。测试 `模型把历史问成这周时仍查全部账本` 就是这个场景的回归。

3.4 模型漏发工具时的补偿与工具优先级

模型不是每次都乖乖调工具。控制器为此留了两道保险：

保险一：计划补偿。 如果本轮 `plannerAllowed` 仍为真，而模型这一轮没给出任何工具调用，控制器会自己跑一次 `plan(userText, today, chartMonth)`；非空就记一条 `decision(model_omitted_tool_call)`，然后直接按计划执行（`AssistantController.java:279-290`）。注意它会把 `plannerAllowed` 置 false，所以一轮里最多补偿一次，不会和模型互相抢。

保险二：优先级排序。 模型可能一口气给多个工具调用，执行顺序由 `rank` 决定：`query_spend` 0、`daily_fortune` 1、`propose_add_bill` 2、`request_diary_context` 3、其它 9（`AssistantController.java:486-500`）。排序的意义是「先查账再说话」：如果一句话既在问账又在记账，查账先跑，`ledgerQueried` 先置真，后面的文本才可能带数字（§3.5）。

这两道保险的分工是：补偿解决「模型偷懒」，排序解决「模型贪心」。测试 `planner_同句先查再记且运势不抢记账` 与 `demo_模型不调工具时仍按计划查账并等待确认` 分别覆盖两者。

3.5 应用侧数字闸门（ReplyGate）

这是整套意图控制里唯一「宁可错杀」的地方。规则只有一行：

```
return needsLedger && !queried && hasFigure(text);
```

（`ReplyGate.java:11-13`。）

读法是：如果本轮「必须查账」（`needsLedger` 来自 `isQuery`，`AssistantController.java:132`），而账还没查成功（`ledgerQueried=false`），那么任何带数字/¥/元的文本都不许上屏。`hasFigure` 的检测范围是 `¥`、`元`、以及任意数字（`ReplyGate.java:15-28`）。闸门在三个地方生效，缺一不可：

位置	作用	代码
流式回调	逐字上屏前拦，避免「先显示再删」	`AssistantController.java:262-265`
`show()`	非流式的整段文本	`AssistantController.java:419-431`
`showOnly()`	整段替换（查账结果摘要）	`AssistantController.java:433-439`

为什么值得「错杀」？因为用户对账单类 App 的容忍度不对称：少显示一句没事，显示一个错金额会被记住很久。这条规则的代价是「模型寒暄里带一个数字也会被吞」，所以配套的设计是：查账成功后 `ledgerQueried=true`，同一轮后续文本不受影响；查账失败则整轮都不显示数字，只显示「这次没有查成账本。」（`AssistantController.java:319-330`）。测试 `历史账单在查到之前不展示数字且包含导入记录`、`replyGate_查账成功前拦住带数字的回复` 覆盖了正反两面。

3.6 提示词与上下文的组装顺序

system 消息不是一段静态文本，而是三段拼起来的，顺序固定（`AssistantController.java:460-469`）：

1. 协议正文：`docs/assistant/SYSTEM_PROMPT.v0.1.md` 抽出正文，作为第一条 system。
2. 上一会话要点（可选）：仅当界面注入了记忆时追加，且附一句「只使用其中已经写出的事实，不要假装记得没有写入的内容。日记仍须先请求确认。」
3. 对话历史：`messages` 里已有的 user/assistant/tool 轮。

工具表单独走请求参数：`streamer.stream(withSystem(), docs.toolsArray(), observer)`（`AssistantController.java:252`），且当 `toolsArray` 为空或 `"[]"` 时不塞 `tools` 字段（`DeepSeekStreamClient.java:53-55`）。这样「无协议也能跑」：`AssistantDocuments.load` 失败时用 `FallbackDocs`，system prompt 为空、tools 为 `[]`（`AssistantHost.java:132-152`），控制器不会因为缺资产文件而崩，只是退化成普通对话。

提示词里的硬规则与代码是互指的，不是两套说法。例如 `SYSTEM_PROMPT.v0.1.md:17-18` 写「数字只能来自本轮 query_spend」「查账不弹确认卡」，对应的代码分别是 §3.5 的闸门与 `beginConfirm` 只处理两类工具（`AssistantController.java:310-315`）；`:20` 写「period=all」对应 `spendArgs` 的修正；`:22-23` 写周口径与「当月=选中月」对应 `WeekRange` 与 `chart_month`。这种「文档说一条、代码有一条、测试守一条」的三点对齐，是本项目控制 AI 生成代码的主要手段：任何一条规则被改坏，至少有一个测试会红。

3.7 同一句话的七种走向

「模型不听话怎么办」是评审最常问的问题。与其解释，不如把同一句「这周食堂花了多少」在七种条件下的走向并排列出——每一行都能在代码里对上，且都有测试或确定性文案：

走向	触发条件	应用侧动作	用户看到	证据
正常查账	模型首轮返回 `query_spend`	`handleCalls` → `SpendQuery` → `ledgerQueried=true` → 二轮汇总	状态「调用工具」→ 球 40 → 汇总与明细	`AssistantController.java:301-343`

走向	触发条件	应用侧动作	用户看到	证据
模型漏发工具	首轮无 `tool_calls`，但 planner 命中	记 `decision(model_omitted_tool_call)`，用计划补一次	与正常查账相同，只多一轮	`AssistantController.java:279-290`
模型抢答数字	需要查账、还没查、正文含 ¥/元/数字	`ReplyGate.withhold` 丢弃这段正文，不显示	先看到「调用工具」，再看到真实结果	`AssistantController.java:262-265`、`ReplyGate.java:11-13`
查账失败	`query_spend` 返回 `error`	记 `toolResult("error")`，回固定文案	「这次没有查成账本。」	`AssistantController.java:319-330`
无 Key	`hasApiKey` 为假	`fail(API_KEY)` → `ERROR` 档	「密钥无效或未授权，请在 DeepSeek 密钥向导里重新测通。」	`AssistantController.java:138-141`、`FailureClassifier.java:21-29`
余额或配额不足	HTTP 402 或响应体含 quota/余额	`fail(QUOTA)` → `ERROR` 档	「DeepSeek 余额或配额不足……」	`FailureClassifier.java:9-19`
用户取消	确认卡可见时点取消	不写库，记 `confirm_record(accepted=false)`	「记账要先请你在卡片上确认，我还没有写入。」	`AssistantController.java:210-241`

七种走向有一个共同点：**用户不会看到「应用在等模型自觉」**。要么拿到结果，要么拿到确定的错误文案，没有第三种「沉默」。

4 工具、确认与安全约束

4.1 四个工具与 schema 约束

工具表固定四个：`query_spend`、`propose_add_bill`、`request_diary_context`、`daily_fortune`（`docs/assistant/tools.schema.v0.1.json:22,73,118,140`）。「塔罗」不是第五个工具，而是 `daily_fortune` 的 `style=tarot_lite`（`AssistantCoreTest.java:28-472` 的 `schema_正好四个工具且塔罗不是独立工具` 在守这条）。

工具	用途	关键参数	是否写库	是否需要确认
`query_spend`	查账并汇总	`period`、`type`、`merchant_contains`、`category`、`start_date`、`end_date`	否	否
`propose_add_bill`	提出一笔记账建议	`amount`、`type`、`date`、`category`、`merchant`、`remark`	否（由应用在确认后写）	是
`request_diary_context`	请求引用本机日记	`purpose`（greeting/tone_adjust）	否	是
`daily_fortune`	生成运势/晴雨/塔罗文案	`style`（fortune/mood/tarot_lite）	否	否

schema 里有一条容易被忽略的约束：`propose_add_bill` 的说明明确「不得直接写 Room」（`tools.schema.v0.1.json:73` 一带），而 `adapter_contract` 又把「应用侧负责落库与回读」写死（`:160-198`）。这不是注释，是控制器实现的前提：模型能做的最大动作是「提建议」，落库是应用的事。

参数解析也做了防御：`parseArgs` 解析失败返回空 `JSONObject`，后续 `optString` 走默认值（`AssistantController.java:502-508`）；`daily_fortune` 的 `style` 只接受三个白名单值，其它一律按 `fortune` 处理（`AssistantController.java:376-391`）。所以模型给一个不存在的工具名，只会得到 `{"error":"unknown tool"}`，不会触发副作用。

4.2 写路径确认门（从 draft 到回读一致）

记账是整个系统唯一会改用户数据的路径，因此被切成「解析 → 校验 → 弹卡 → 写入 → 回读 → 比对 → 回执」七段，每段都有失败出口。

第一段，解析。 `BillDraft.parse` 把模型参数转成草稿，并给缺失字段兜底：空分类→「其他」，非法日期→今天，`HH:mm` 补成 `HH:mm:ss`、非法时间→`12:00:00`（`BillDraft.java:30-58`）。兜底的原则是「能不打断就不打断」，但金额不兜底。

第二段，校验。 `valid()` 要求 `amount >= 0.01` 且日期形如 `yyyy-MM-dd`（`BillDraft.java:60-62`）。不合法时不弹卡：控制器写一条 `status=cancelled` 的工具消息，记 `toolResult("missing amount")`，然后回「还缺金额，我没有写入。」（`AssistantController.java:359-367`）。这一步是「先问再记」规则的落点（`SYSTEM_PROMPT.v0.1.md:24`）。

第三段，弹卡。 `beginConfirm` 设置 `pending`，状态切到 `AWAITING_CONFIRM`，回调 `onConfirmCard("确认记下这一笔？", draft.cardBody())`（`AssistantController.java:371-372`）。卡片正文只有类型、金额、分类/商户/日期/备注，空值显示「—」（`BillDraft.java:91-98`），不夹带模型生成的解释性文字。

第四段，写入。 用户点确认后 `doConfirm` 才执行 `ledger.insertAndRead(draft.toBill())`，`batchId=0`，并用自增 id 回读（`AssistantController.java:197`，`RoomBillLedger.java:37-50`）。

第五段，比对。 `draft.matches(saved)` 按「分」比较金额，并逐字段比 `type/category/merchant/remark/date`（`BillDraft.java:100-110`）。比对结果决定回执话术：`matched ? "已写入并回读：" : "已写入，但回读和确认卡不一致："`（`AssistantController.java:523-531`）。

第六段，回执。 工具消息带 `status=confirmed_and_verified` 或 `mismatch`，并附 `bill` JSON（`AssistantController.java:201-204`）。这条消息会进对话历史，所以模型下一轮「知道」写成功了没有。

第七段，取消。 `doCancel` 不写库，记 `confirmRecord(action, false, ...)`，回「记账要先请你在卡片上确认，我还没有写入。」（`AssistantController.java:221-241`）。

这七段的价值在于「回读」不是可选项：如果 `insert` 成功但回读为空，回执会变成「确认后没有读回这条记录。」（`AssistantController.java:523-531`），而不是继续宣称写入成功。测试 `demo_确认前不写库确认后回读一致` 与 `demo_取消确认不写库` 分别守正反两条。

4.3 密钥与隐私边界

密钥只有一条路径：`DeepSeekKeyStore`。它用 Android Keystore 里的 AES 密钥做 GCM 加密，密文与 IV 存 SharedPreferences（`deepseek_keystore` / `ciphertext` / `iv`），别名 `campusbill.deepseek.aes`，GCM tag 128 位（`DeepSeekKeyStore.java:21-28`）；`get()` 先解密，失败返回空串，再尝试旧的明文键 `ai_config/api_key` 并自动迁移（`:37-70`）。`AiConfig` 里只留接口地址与模型名，保存设置时会主动 `remove` 旧的明文键。

权限面也很窄：Manifest 只有 INTERNET 一条权限（`AndroidManifest.xml:5`），且 `allowBackup="false"`（`:8`）。前者说明没有读通讯录/定位/存储的越界，后者说明 Keystore 密文不会随备份离开设备。工具表里的 `adapter_contract` 另有一条「密钥不进日志、不进仓库」（`tools.schema.v0.1.json:160-198` 一带），由 §4.4 的脱敏实现兜底。

需要诚实说明的边界：Keystore 的安全性依赖设备锁屏与系统实现，本册没有做侧信道或 root 场景的评估；`get()` 解密失败时静默返回空串，用户只会看到「没有 Key」的失败态，不会看到密钥损坏的具体原因。这两点都写进 §7.2。

4.4 过程留痕与脱敏

每轮对话会写一份 JSONL，文件名 `yyyy-MM-dd-HHmmss.jsonl`（Asia/Shanghai），目录 `filesDir/assistant/process/runtime`（`FileProcessLog.java:20-31`）。写入失败不打断对话（`:47-49`），因为留痕是旁路，不是主链路。

事件类型与 `think-protocol.v0.1.md` 对齐，共七种：

事件	字段	用途
`status`	`ts,type,value`	四档状态迁移
`prompt`	`ts,type,version,hash`	本轮用了哪版提示词（hash 为正文 SHA-256）
`think`	`ts,type,text`	思考摘要（只进日志，不上屏）
`tool_call`	`ts,type,name,arguments`	工具名与参数（先脱敏再尝试 JSON 化）
`tool_result`	`ts,type,name,result_summary`	结果摘要（如 `total=… count=…`）
`confirm_record`	`ts,type,action,accepted,preview,verified`	确认/取消与回读校验
`decision`	`ts,type,text,planned`	应用侧决策（如 `model_omitted_tool_call`）

脱敏发生在两个位置：`ProcessEvents` 的 `line()` 对整条记录统一过一遍 `SecretRedactor`（`ProcessEvents.java:98-100`），`toolCall` 的 `arguments` 额外先脱敏再解析（`:41-57`）。规则是两条正则：`sk-[A-Za-z0-9_\-]+` → `sk-***`，`(?i)bearer\s+[A-Za-z0-9\-._~+/]+=*` → `Bearer ***`（`SecretRedactor.java:11-15`）。测试 `process_事件不留下密钥` 是这条路径的回归。

`FileProcessLog` 与 `ProcessEvents` 的分工值得注意：前者只管「写文件、不抛异常」，后者只管「事件长什么样、怎么脱敏」。控制器两边都不认识，只调 `ProcessEvents.xxx()` 并把结果交给 `log`。这样换存储位置（比如改成上传）不需要动事件定义。

4.5 失败分类与四档状态的封闭性

DeepSeek 的 HTTP 结果被收成三类，不允许出现第四类（`FailureClassifier.java:9-19`）：

触发	分类	用户看到的文案
401 / 403	`API_KEY`	密钥无效或未授权，请在 DeepSeek 密钥向导里重新测通。
402，或响应体含 `insufficient` / `quota` /余额/欠费	`QUOTA`	DeepSeek 余额或配额不足，请到服务商控制台检查。
其它（网络、5xx、解析失败）	`NETWORK`	网络异常，这次没有连上 DeepSeek。

三类的共同点是「都能在应用内自愈或明确告知」：密钥类引导去向导重填，配额类引导去控制台，网络类建议重试。`statusLine` 里刻意不写「重试」二字（`FailureClassifier.java:31-40`），因为重试按钮由界面单独给，不靠文案暗示。

失败后的状态迁移只有一条：清 `pending` → `machine.set(ERROR)` → 收起确认卡 → 记 `status` 事件 → 显示一行文案（`AssistantController.java:393-402`）。没有「半失败」状态，也没有把错误塞进 `AWAITING_CONFIRM` 的写法。测试 `network_出错时收起确认卡`、`missingKey_是出错而不是新状态`、`failure_只把密钥配额和网络标成出错` 三条一起守这个封闭性。

一个可复现的边界行为：没有 Key 时发「这周食堂花了多少」，`AiChatActivity` 仍会把它送进控制器（因为 `assistantUtterance=true`），控制器在 `doStart` 里立刻 `fail(API_KEY, "请先在 DeepSeek 密钥向导里填写并测通密钥。")`（`AssistantController.java:138-141`）。所以无 Key 机器上的预期是「明确的错误态」，不是「沉默」也不是「本地瞎答」——这正好让评审在没有密钥的情况下也能验证失败路径。

4.6 参数边界值：坏输入怎么收场

工具参数来自模型，不能假设它总是合法。下表是本轮读代码逐个确认的边界处理，每行都能在源码里对上；原则是「要么按默认值继续，要么明确拒绝，绝不半写」：

输入	边界处理	代码
`amount` 缺失、为 0 或负数	`valid()` 要求 ≥ 0.01；不弹卡，回「还缺金额，我没有写入。」	`BillDraft.java:60-62`、`AssistantController.java:359-367`
`amount` 有浮点误差	回读比对按分取整（`Math.round(amount*100)`），不比 double	`BillDraft.java:100-110`
`date` 缺失或非法	兜底为今天，不阻断	`BillDraft.java:44-46`
`time` 缺失或非法	规范成 `HH:mm:ss`；仍非法则 `12:00:00`	`BillDraft.java:48-52`
`category` 为空	兜底为「其他」	`BillDraft.java:40-42`
`style` 不在白名单	只接受 `fortune` / `mood` / `tarot_lite`，其余按 `fortune`	`AssistantController.java:376-391`
`period=custom` 缺 `start_date` / `end_date`	返回 `{"error":"custom 需要 start_date 与 end_date"}`，不返回数字	`SpendQuery.java:65-82`
工具名不存在	返回 `{"error":"unknown tool"}`，不产生副作用	`AssistantController.java:376-391`
`arguments` 不是合法 JSON	解析失败返回空 `JSONObject`，后续按默认值走	`AssistantController.java:502-508`

这张表也说明了「为什么金额单独严格」：日期、分类、时间错了只是记到别处，金额错了会让账本对不上，所以只有金额是硬校验，其余都是兜底。

5 可复现性论证

5.1 环境与前置条件

本节的环境是本轮实测所用的机器。表格里的版本号都取自命令输出，不是「大概」。

项	值	取得方式
操作系统	Windows 11 家庭版 中文版，NT 10.0.26200	`[System.Environment]::OSVersion.VersionString`
JDK	21.0.12.1 LTS（2026-08-18）	`java -version`
构建工具	Gradle wrapper 9.6.0（`gradle-9.6.0-bin.zip`，networkTimeout 10000）	`gradle/wrapper/gradle-wrapper.properties`
Android SDK	`C:\Users\ZXM\AppData\Local\Android\Sdk`	`local.properties` 的 `sdk.dir`
adb	不在 PATH，需用 `%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe`	`adb version` 报「无法识别」
文档渲染	Edge 157.0.4341.46（headless 打印 PDF）	`msedge.exe` 的 ProductVersion
文档后处理	Python 3.11.9 + pypdf 6.21.2 + PyMuPDF	`python -c "import pypdf, fitz"`
未安装	pandoc、LibreOffice（`soffice` 不存在）	`Get-Command pandoc/soffice`

「未安装」这一行不是抱怨，而是设计的约束：渲染链路必须只依赖 Windows 自带能力 + Python 两三个包，否则评委会在「装 LibreOffice」这一步卡住。§5.4 会说明这条链路具体怎么走。

5.2 构建、安装与测试命令

以下命令都在仓库根目录执行，按顺序即可复现本册的数字。

（1）单元测试（JVM，不需要模拟器）：

```
.\gradlew.bat :app:testDebugUnitTest
```

预期：`BUILD SUCCESSFUL`，结果 XML 落在 `app/build/test-results/testDebugUnitTest/`。用下面这段脚本把「类数/用例数/失败数」复算出来，而不是看控制台摘要：

```
python - <<'PY'
import glob, xml.etree.ElementTree as ET
t=f=e=s=0; classes=0
for p in glob.glob('app/build/test-results/testDebugUnitTest/*.xml'):
    r=ET.parse(p).getroot(); classes+=1
    t+=int(r.get('tests')); f+=int(r.get('failures')); e+=int(r.get('errors')); s+=int(r.get('skipped'))
print('classes',classes,'tests',t,'failures',f,'errors',e,'skipped',s)
PY
```

本轮的输出是 `classes 8 tests 51 failures 0 errors 0 skipped 0`，与 §1.3 一致。各测试类的分布见 §6.2。

（2）打 Debug 包：

```
.\gradlew.bat :app:assembleDebug
```

产物 `app/build/outputs/apk/debug/app-debug.apk`，本轮 26,702,585 字节，SHA-256 见 §1.3。注意 `assemble` 会先触发 `syncAssistantDocs`（`app/build.gradle:34-42`），所以 assets 里的协议文档与 docs 源一致是构建保证，不是手工同步的结果。

（3）安装到设备（可选）：

```
%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe install -r app\build\outputs\apk\debug\app-debug.apk
```

（4）确认基线没有被本册改动污染：

```
git rev-parse HEAD
git status --porcelain
```

本册只新增三个文件（Markdown、PDF、渲染脚本），不会修改任何既有源码或文档；git status 的差集见 §6.2 的「新增文件」一栏。

5.3 演示数据与可核对数字

演示数据在 `docs/assistant/DEMO-SCRIPT.md` 的 §1.4–§1.5：四笔账单（周一食堂 18、周二食堂 22、周三校园超市 120、周四话费 50），本月合计 ¥210.00，本周食堂 2 笔 ¥40.00。预算口径分两种状态，别混：干净安装（0 笔）但已设预算 1500 时，首页显示总预算 ¥1500.00、预算剩余 ¥1500.00（`MainActivity.java:615-620` 的 `BUDGET_OK` 分支，因为 `monthExpense=0`）；只有未设预算才显示「未设置预算」（`MainActivity.java:603-608`、`strings.xml:18`）。导入四笔后剩余 ¥1290.00，且「与上月消费对比」显示 ↑ ¥210.00；演示中再记两笔 18 元，剩余依次变为 1272、1254。「暂无账单数据，请先导入账单」是空账本时「与上月消费对比」位置的文案（`MainActivity.java:227`、`strings.xml:219`），不是预算剩余。

检查点	期望	口径
干净安装 + 预算 1500	总预算 ¥1500.00 / 剩余 ¥1500.00	0 笔、`monthExpense=0` 走 `BUDGET_OK`（`MainActivity.java:615-620`）
本月总支出	¥210.00	4 笔支出之和；本地 `ReplyGenerator`
本周食堂	2 笔 / ¥40.00	周一+周二，`merchant_contains` 与 `category` 同时给
预算剩余	¥1290.00	1500 − 210
与上月消费对比	↑ ¥210.00	上月 0、本月 210（`strings.xml:219` 只在空账本时出现）
记第一笔 18 后	¥1272.00	确认卡回执「已写入并回读」
记第二笔 18 后	¥1254.00	同上

必须说明证据等级：这些数字来自演示脚本的既定流程，本轮**没有**在真机上重放整条演示（原因见 §7.2）。本册能担保的是：产生这些数字的代码路径（`SpendQuery` 的分累加、`RoomBillLedger` 的一本账、`BillContextBuilder` 的当月口径）都有单元测试覆盖，且 §6 给出了代码锚点；不能担保的是「设备上每一帧都按脚本发生」，那需要录屏或仪器化测试，属于后续工作。

5.4 本文档自身的构建与页数自检

本册不是手工排版，而是由 `docs/assistant/tech-doc/render.ps1` 生成的。一条命令：

```
powershell -ExecutionPolicy Bypass -File docs\assistant\tech-doc\render.ps1
```

脚本做的事，按顺序：

1. 读 `TECHNICAL-DOC.md`，用内置的 Python 转换器把 Markdown 转成 HTML（标题/表格/代码块/引用/列表/行内强调）。
2. 生成目录：扫描 h1/h2，给每节分配锚点，先填占位页码 `00`。
3. 用 `msedge.exe --headless=new --disable-gpu --no-pdf-header-footer --print-to-pdf` 打印第一版 PDF。
4. 用 PyMuPDF 在 PDF 里逐页搜标题文本，得到每节真实起始页，回填目录页码。
5. 再打印第二版，用 pypdf 复核「回填后页数没有变化」，再盖页脚（`第 N 页 · 共 M 页`）。
6. 用 pypdf 读页数、字节数、SHA-256，打印一行 JSON 统计；若页数不在 24–30，脚本以非零退出。

需要说明可复现的粒度：PDF 的二进制 SHA-256 每次都会变（Edge 打印会在文件里写入创建时间与随机 `/ID`），稳定的是**页数、字节数与全文文本指纹**。脚本末尾除 SHA-256 外还会打印一行 `text_sha`，它是对全文抽取文本算的哈希；要和本册核对，请比页数与 `text_sha`，而不是比二进制哈希。

版式按评审硬条件设定并实测：A4，正文 10.5pt（行高 1.62），左右页边距 18.5mm、上 18.6mm、下 20mm（实测正文四边均 ≥18mm）；表格 8.85pt、代码 8.25pt 以容纳列宽。页码在页脚区（距底 8mm），正文下沿始终留 20mm。

字体不依赖系统是否装了中文字体：HTML 里显式写 `font-family: "Microsoft YaHei", "SimHei", sans-serif`，页脚用 `simhei.ttf` 嵌入，PDF 的 Font 字典里可以看到嵌入子集。这样评委在自己的机器上打开时不会跳字体。

页数自检的意义是：`SUBMISSION-CHECKLIST.md:26` 要求不超过 30 页，而「不超过」是硬条件。把页数检查写进脚本，就不需要人肉数；§1.3 的 APK 摘要与测试计数也是同理——能自动算的，不手填。

5.5 不联网可复现 / 需联网才能复现的边界

复现对象	是否需要网络	是否需要设备	本册是否给证据
构建 APK、跑 51 个单测	否（依赖已在 Gradle 缓存）	否	是（§1.3、§6.2）
生成本 PDF	否	否	是（§5.4、§6.3）
安装、启动、导入演示数据	否	是（Android 7.0+）	否（§7.2 未实测）
无 Key 的错误态	否	是	代码锚点（§4.5）
真实 DeepSeek 对话质量	是（需 Key 与余额）	是	否（§7.2 未实测）

这张表的读法是：**能离线验证的部分，本册都给了可复算的证据；需要 Key 或设备的部分，本册只给代码路径与预期，不冒充已测。** 这样评审可以自由选择验证深度，而不会在「按文档做却发现数字对不上」时怀疑整份文档。

5.6 复现失败时的排查矩阵

照着 §5.2 跑却对不上时，先查这张表，再怀疑文档：

现象	最可能的原因	怎么确认	对应章节
`gradlew` 报 SDK location not found	`local.properties` 缺 `sdk.dir`	读 `local.properties`；Android Studio 首次打开会自动生成	§5.1、`REPRODUCE.md` §1
单测数量不是 51	Gradle 复用了旧结果	`gradlew testDebugUnitTest --rerun-tasks`，再看 XML 的 `timestamp`	§5.2、§6.2
无 Key 时对话一直「思考」不结束	走的是旧构建，或未进入控制器的本地分支	状态应停在「出错」；对照 `AssistantController.java:138-141`	§4.5
问本月总支出，回复只有一句「我看看」	句子不在关键词表，planner 未命中	换成快捷芯片原句「本月总支出」再试	§3.3
说「记一笔」却不弹确认卡	金额缺失或小于 0.01	回复应是「还缺金额，我没有写入。」；对照 `AssistantController.java:359-367`	§4.2
记完账首页数字没变	只发了建议、没点确认（取消不写库）	确认卡出现前首页不应变化；点确认后才写入并回读	§4.2
PDF 页数不在 24–30	正文增删后页数漂移	脚本以非零退出；按需调 `-MinPages` / `-MaxPages` 或增删内容	§5.4

排查顺序建议从「命令是否真的跑了」开始：Gradle 的 up-to-date、脚本临时目录的写权限，都会造成「看起来跑了、其实没跑」。

6 证据索引

6.1 代码锚点总表

下表是本册引用过的全部代码锚点（文件:行号），按主题排序。行号以 HEAD `910041e` 为准（代码树同 `966f41c`）；若后续提交改动，行号会漂移，但类型与方法名不变，仍可用 `grep` 定位。

编号	锚点	说明	章节
A01	`AiChatActivity.java:283-284`	助手计划非空即交给控制器	§3.1
A02	`AiChatActivity.java:285-286`	旧意图未命中且有 Key 才走在线兜底	§3.1
A03	`AiChatActivity.java:287-296`	两条路由任一为真 → `startOrIgnore`	§3.1
A04	`AiChatActivity.java:306-422`	本地离线意图 dispatch（六个分支）	§3.1
A05	`AiChatActivity.java:424-452`	在线兜底：账本摘要注入 + 失败降级	§3.1、§5.5
A06	`AiChatActivity.java:73`	旧类注释「全程不联网」与现状不一致	§7.1
A07	`strings.xml:142-145`	四个快捷芯片文案	§3.2
A08	`AssistantController.java:55`	`MAX_ROUNDS = 4`	§2.2、§7.3
A09	`AssistantController.java:124-143`	`doStart`：初始化本轮上下文	§2.2
A10	`AssistantController.java:135-136`	`prompt` 事件记版本与提示词 SHA-256	§2.5、§4.4
A11	`AssistantController.java:138-141`	无 Key → `API_KEY` 失败态	§3.1、§4.5
A12	`AssistantController.java:262-265`	流式正文过 `ReplyGate`	§3.5
A13	`AssistantController.java:274-277`	模型给出工具调用则关闭补偿	§3.4
A14	`AssistantController.java:279-290`	模型漏发工具时的计划补偿	§3.4
A15	`AssistantController.java:301-343`	`handleCalls`：工具执行主循环	§2.2、§3.4
A16	`AssistantController.java:310-315`	确认类工具 → `beginConfirm` 挂起本轮	§2.2、§4.2
A17	`AssistantController.java:319-330`	查账成功置 `ledgerQueried` 并显示摘要	§3.5
A18	`AssistantController.java:345-374`	`beginConfirm`：日记/记账两条确认路径	§4.2
A19	`AssistantController.java:359-367`	金额缺失 → 不弹卡，明确「没有写入」	§4.2
A20	`AssistantController.java:174-208`	`doConfirm`：写入→回读→比对→回执	§4.2
A21	`AssistantController.java:197-205`	`insertAndRead`、`matches`、`verifySentence`	§4.2
A22	`AssistantController.java:221-241`	`doCancel`：不写库的固定话术	§4.2
A23	`AssistantController.java:393-402`	`fail`：清挂起 → ERROR → 收起确认卡	§4.5
A24	`AssistantController.java:419-439`	`show` / `showOnly` 同样过闸门	§3.5
A25	`AssistantController.java:441-450`	无显式时间窗时强制 `period=all`	§3.3
A26	`AssistantController.java:460-469`	system 三段拼装 + 记忆注入	§3.6
A27	`AssistantController.java:480-500`	工具排序 `rank`	§3.4
A28	`AssistantController.java:523-531`	回执话术（一致/不一致/没读回）	§4.2
A29	`AssistantIntentPlanner.java:41-68`	计划：查账→记账→日记→运势	§3.2
A30	`AssistantIntentPlanner.java:70-76`	`needsSpendQuery` 即 `isQuery`	§3.5
A31	`AssistantIntentPlanner.java:82-87`	无显式时间窗 → 全部账本	§3.3
A32	`AssistantIntentPlanner.java:89-113`	`isQuery` 的判定顺序	§3.2
A33	`AssistantIntentPlanner.java:116-119`	本地快捷句白名单	§3.2
A34	`AssistantIntentPlanner.java:121-124`	「怎么导入/怎么记账」留在本地	§3.2
A35	`AssistantIntentPlanner.java:208-231`	查账参数推导（五种时间窗）	§3.3
A36	`AssistantIntentPlanner.java:233-256`	记账参数推导（分类/商户）	§3.2
A37	`ReplyGate.java:11-13`	数字闸门的一行规则	§3.5

编号	锚点	说明	章节
A38	`ReplyGate.java:15-28`	`hasFigure` 检测 ¥/元/数字	§3.5
A39	`SpendQuery.java:58-99`	时间窗、类型、关键词过滤与排序	§2.4
A40	`SpendQuery.java:101-105`	以「分」累加后回浮点	§2.4
A41	`SpendQuery.java:121-141`	返回 JSON 结构与 20 条明细上限	§2.4
A42	`RoomBillLedger.java:22-29`	一本账：导入与手动都参与统计	§2.4
A43	`RoomBillLedger.java:37-50`	写入后按 id 回读	§2.4、§4.2
A44	`BillContextBuilder.java:24-26`	最近 15 条、Top5 上限	§3.6
A45	`BillContextBuilder.java:137-184`	上下文摘要组装口径	§3.6
A46	`BillDraft.java:30-58`	草稿解析与缺省兜底	§4.2
A47	`BillDraft.java:100-110`	回读比对按分比较	§4.2
A48	`AssistantHost.java:46-78`	依赖装配与主线程回调	§2.1
A49	`AssistantHost.java:94-107`	`busy` CAS 防并发	§2.3
A50	`AssistantStatus.java:9-12`	状态只有四档	§2.3
A51	`AssistantStatusMachine.java:24-26`	确认卡只在等待确认可见	§2.3
A52	`EmotionHooks.java:26-30`	球的默认 id 与协议覆盖	§2.3
A53	`EmotionHooks.java:80-109`	场景到球的映射	§2.3
A54	`FileProcessLog.java:25-31`	JSONL 目录与文件名	§4.4
A55	`FileProcessLog.java:37-51`	写入失败不打断对话	§4.4
A56	`ProcessEvents.java:41-57`	`tool_call` 先脱敏再序列化	§4.4
A57	`SecretRedactor.java:11-15`	两条脱敏正则	§4.4
A58	`DeepSeekKeyStore.java:21-28`	Keystore 别名、偏好键、GCM 位数	§4.3
A59	`DeepSeekKeyStore.java:37-70`	解密读取与旧明文键迁移	§4.3
A60	`FailureClassifier.java:9-19`	HTTP → 三类失败	§4.5
A61	`FailureClassifier.java:21-29`	三类用户文案	§4.5
A62	`DeepSeekStreamClient.java:59-60`	连接 10s / 读取 60s	§2.3
A63	`AssistantDocuments.java:33-41`	启动时加载四份协议	§2.5
A64	`AssistantDocuments.java:48-58`	只从 assets 读，UTF-8	§2.5
A65	`app/build.gradle:34-42`	`syncAssistantDocs`：docs 是唯一源	§2.5、§5.2
A66	`AppDatabase.java:12-14`	Room version 3 与实体表	§2.4
A67	`AndroidManifest.xml:5`	只有 INTERNET 权限	§4.3
A68	`AndroidManifest.xml:8`	`allowBackup="false"`	§4.3
A69	`AssistantCoreTest.java:28-472`	26 个核心行为测试	§2.2、§4
A70	`app/build.gradle:59-60`	测试依赖 junit 4.13.2 / org.json	§5.2
A71	`tools.schema.v0.1.json:198`	`app_gate` 约束	§4.1
A72	`SYSTEM_PROMPT.v0.1.md:17-20`	数字来源、确认卡、period=all	§3.6
A73	`intent-control.v0.1.md:7`	「本轮没查到账不展示数字」	§3.5
A74	`MainActivity.java:603-620`	四态预算：未设置 / 超支 / 正常	§5.3
A75	`strings.xml:18`、`strings.xml:219`	「未设置预算」与空账本列表文案	§5.3
A76	`MainActivity.java:227`	空账本时「与上月消费对比」用提示文案	§5.3、§5.6

6.2 构建、测试与产物证据

证据	取得命令	结果
提交基线	`git rev-parse HEAD`	`910041eda1e70afe846bd3492167025d7fe8f667`（代码树同 `966f41c`）

证据	取得命令	结果
单元测试类数	解析 XML（§5.2 脚本）	8
单元测试用例	同上	51（AiReplyTest 6、BillAnalysisUtilsTest 2、AssistantCoreTest 26、SessionRulesTest 5、AttitudeGazeMathTest 2、ChatBallFadeTest 3、ChatBallMetricsTest 4、ChatInputChromeTest 3）
失败/错误/跳过	同上	0 / 0 / 0
测试时间	同上（`timestamp` 属性）	2026-11-12T05:52:11.497Z–05:52:11.634Z
APK 字节	`(Get-Item app-debug.apk).Length`	26,702,585
APK 摘要	`Get-FileHash -Algorithm SHA256`	`B5A4B94991897FCCFB08C6FCF652F5FC0651615909DACE4C9AAD053E29B6CFC6`
助手包规模	脚本统计	30 文件 / 2,391 行
本册新增文件	`git status --porcelain docs/assistant`	`?? docs/assistant/TECHNICAL-DOC.md`、`?? docs/assistant/TECHNICAL-DOC.pdf`、`?? docs/assistant/tech-doc/render.ps1`

「新增文件」一栏的写法是刻意的：本册只新增这三个文件，不修改任何既有文件。评审可以用这三个路径与 `git status` 对照，确认文档没有夹带代码改动。

6.3 协议文档清单

`docs/assistant` 下的协议与说明文档（字节数 / 行数，本轮实测），它们是运行时输入或评审材料，不是装饰：

文件	字节	行数	作用
`SYSTEM_PROMPT.v0.1.md`	5,377	44	system 正文与硬规则
`tools.schema.v0.1.json`	7,707	203	四个工具的 schema 与 adapter 契约
`intent-control.v0.1.md`	4,947	55	意图优先级、示例与反例
`think-protocol.v0.1.md`	3,152	61	四档状态与七类 JSONL 事件
`emotion-map.v0.1.md`	9,943	106	球 id 与 `hook:` 协议行
`session.v0.1.md`	3,690	41	会话记忆约定
`DEMO-SCRIPT.md`	41,557	518	演示分镜、话术、数据
`REPRODUCE.md`	22,483	149	环境、依赖、复现主线
`SUBMISSION-CHECKLIST.md`	2,907	34	交付清单与硬性要求
`README.md`	6,839	25	目录导览
`THIRD_PARTY.md`	4,821	52	第三方依赖与许可

6.4 单元测试清单（26 个方法）

`AssistantCoreTest.java:28-472` 的 26 个方法不是凑数的覆盖，每个都钉住一条上文的断言。列在这里，方便评审把「51 tests / 0 failures」拆回到具体边界：

#	测试方法	守的边界
1	`status_只有四档且确认卡只在等待确认`	四档封闭
2	`schema_正好四个工具且塔罗不是独立工具`	工具数固定
3	`systemPrompt_只取正文且不含给作者的说明`	正文抽取
4	`week_本周是周一到周日`	周口径
5	`querySpend_食堂按分类或商户命中且只统计支出`	食堂专名
6	`当月跟随首页选中月_点名月份用那个月`	chart_month
7	`querySpend_thisMonth按选中月而不是本周`	月度≠本周
8	`querySpend_all含导入与手动且合计不因明细截断`	一本账

#	测试方法	守的边界
9	`planner_未指定时间或历史导入用all且快捷句仍本地`	all 兜底
10	`sse_拼出思考正文和分片工具参数`	SSE 解析
11	`failure_只把密钥配额和网络标成出错`	三类失败
12	`planner_同句先查再记且运势不抢记账`	工具优先级
13	`fortune_塔罗只是玩法且同日稳定`	塔罗非工具
14	`diary_未确认不得引用`	日记门
15	`demo_确认前不写库确认后回读一致`	写路径与回读
16	`demo_取消确认不写库`	取消不写
17	`demo_模型不调工具时仍按计划查账并等待确认`	planner 补偿
18	`历史账单在查到之前不展示数字且包含导入记录`	数字闸门
19	`模型把历史问成这周时仍查全部账本`	all 修正
20	`replyGate_查账成功前拦住带数字的回复`	闸门单测
21	`fortune_调度第二个工具且不弹确认卡`	不写库不弹卡
22	`network_出错时收起确认卡`	错误态
23	`missingKey_是出错而不是新状态`	错误态
24	`retry_再发一次失败的流式请求`	重试
25	`process_事件不留下密钥`	脱敏
26	`emotion_四状态有球映射且可被协议钩子覆盖`	球映射

这张表与 §6.2 的统计口径互补：§6.2 证明「跑了多少」，本表说明「跑的是什么」。另有 25 个测试分布在 `AiReplyTest` (6)、 `SessionRulesTest` (5)、 `ChatBallMetricsTest` (4)、 `ChatBallFadeTest` (3)、 `ChatInputChromeTest` (3)、 `BillAnalysisUtilsTest` (2)、 `AttitudeGazeMathTest` (2)，管的是本地回复文案、会话规则与球/输入框的布局计算，与本册主线关系较弱，不逐条展开。

7 已知局限与风险

7.1 已知文档/代码不一致

诚实列出来，不藏着：

1. `AiChatActivity.java:73` **的类注释还写着「全程不联网、不收集隐私数据」。** 这句话在引入 DeepSeek 助手后已经不成立：联网只发生在显式对话路径，但注释没跟着改。代码行为以 §3.1、§4.3 为准，注释属于待修项。之所以不改，是因为本轮任务限定「只新增三个文件」，改源码会破坏交付边界；下一轮应把它改成「默认离线，仅对话路径联网」。
2. `FailureClassifier.java:9-19` **的** `body.contains("quota")` **出现两次。** 属于冗余判断，不影响结果（第二次永假），但读起来像笔误。同样因为不改既有文件而保留，记录在此供后续清理。
3. **演示数字与设备实测的落差。** `DEMO-SCRIPT.md` 的 210/1290/1272/1254 是脚本口径，本轮没有真机重放（§5.3、§7.2）。两者不冲突，但证据等级不同，不能混着引用。

这三条的共同点是：都不影响「能不能跑」，只影响「读起来是否完全一致」。把它们写在最前面，是为了让评审不必替作者发现。

7.2 未实测项（明确标注）

以下项目本轮**没有实测**，本册不给数字、不给百分比、不给「目测良好」：

未实测项	为什么没测	影响
真机端到端演示流程	需要模拟器/设备与导入操作，本轮聚焦文档交付	§5.3 的数字属脚本口径，非本轮重放
真实 DeepSeek 对话质量（措辞、拒答边界）	需要 Key 与余额，且模型版本会漂移	只能担保代码约束，不担保「模型每次都听话」
助手首字延迟 / 整轮耗时	需要多次采样与仪器，单次测量没有意义	不写任何延迟指标，避免伪精度
APK 冷启动、内存占用	同上	不写
Room/SQLite 与 Android Keystore 的真机路径	JVM 单测不覆盖 Android 框架；本轮未跑仪器化测试	`insertAndRead` 的 SQL 行为与 Keystore 解密只有代码与设计证据
5–8 分钟演示视频	视频尚未录制	交付清单第③项待补
PDF 在评委机器上的渲染差异	只在本机 Edge 打印过	已嵌入字体子集降低风险，但未跨机验证
中文口令/方言/错别字的泛化	没有标注语料，无法给出准确率	关键词路由的漏判风险见 §7.3

这张表比任何「性能优秀」的形容词都更有用：它告诉评审哪些结论可以直接采信（构建、测试、产物摘要），哪些必须自己跑一遍（设备行为、模型行为）。

7.3 设计取舍的副作用

每个设计决定都有代价，列出来比藏起来好：

- **关键词路由（正则/子串）而不是小模型分类。** 好处是零延迟、可解释、可测试；代价是漏判同义词（「伙食费」「饭钱」不在词表里），且「预算」是子串命中，导致「这个月预算还剩多少，顺便看看食堂」整句留在本地，只答预算。缓解手段是演示话术收敛到词表内（`DEMO-SCRIPT.md`），根治需要同义词表或分类器，属后续工作。
- **数字闸门只认 ¥/元/数字。** 模型若用中文数字「两百一」描述金额，闸门拦不住。当前提示词要求金额用阿拉伯数字，但没有强制校验；这是一个已知缺口。
- **明细 20 条上限。** 合计不受影响，但用户若要求「把这周的账都列出来」，超过 20 条会被截断且没有「还有 N 条」的提示。`items` 的结构在 `SpendQuery.java:128-140`，加提示是低风险改动。
- **MAX_ROUNDS=4。** 查账+记账+运势的极端组合可能用到第 4 轮；超过就直接收状态（`AssistantController.java:243-247`），不会无限循环。代价是复杂指令可能少说一句收尾。
- **单线程串行。** 好处是没有并发写库与状态竞争；代价是长回答期间用户点确认会排队。因为确认卡本身要求用户动作，实际感知不明显。

- **JVM 单测替代仪器化测试。** 快、可在 CI 跑；代价是 Room/Keystore 的真实行为只能靠设计审查。这是本轮最值得补的一块（§7.4）。

7.4 下一步可验证清单

按「投入/收益」排序，供下一轮直接认领：

1. **补 3 条仪器化测试**：`insertAndRead` 的 Room 路径、Keystore 的存/取/清、确认卡的可见性。验收标准是可重复跑通且不依赖网络。
2. **给路由加同义词表与中文数字识别**，并为「伙食费/饭钱/两百一」各加一条单测；闸门增加中文数字检测。
3. **明细截断提示**：当 `matched.size() > 20` 时在摘要里加「另有 N 条未列出」，单测覆盖。
4. **真机重放演示脚本并录屏**，把 §5.3 的数字从「脚本口径」升级为「实测口径」。
5. **清理 §7.1 的两处注释/冗余判断**（需允许改源码的轮次）。

附录 A 术语表

术语	含义	锚点
计划（plan）	应用侧根据原句推导出的工具调用列表，不依赖模型	`AssistantIntentPlanner.java:41-68`
补偿	模型漏发工具时，应用按计划代跑一次	`AssistantController.java:279-290`
闸门（ReplyGate）	查账成功前拦截带数字文本的应用侧规则	`ReplyGate.java:11-13`
挂起	确认类工具把本轮停在等待确认，不继续跑模型	`AssistantController.java:310-315`
回读	写入后按 id 再查一次，作为回执证据	`RoomBillLedger.java:37-50`
一本账	导入与手动记录共用同一张表，统计不区分来源	`RoomBillLedger.java:22-29`
选中月	首页折线图当前月份，`chart_month` 跟随它	`SpendQuery.java:68-70`
四档	思考/调用工具/等待确认/出错，空闲不算一档	`AssistantStatus.java:9-12`

附录 B 评审速查（3 分钟）

如果只有三分钟，按这个顺序核对，每一步都不需要 Key：

1. **看边界**：`ReplyGate.java:11-13` 一行规则 + `AssistantController.java:138-141` 无 Key 的确定失败态。→ 证明「AI 不能编金额」。
2. **看写路径**：`AssistantController.java:174-208` 的写入→回读→比对，以及 `BillDraft.java:100-110` 的按分比较。→ 证明「AI 不能直接写库」。
3. **看可复现**：跑 §5.2 的两条命令，得到 51/0 与 APK 摘要；或直接看 §1.3 的表。→ 证明数字不是手填。
4. **看诚实**：§7.2 的未实测清单。→ 证明作者知道边界在哪。

三分钟能验证的只有这些；更深的意图控制细节在 §3，架构在 §2，证据总表在 §6.1。

附录 C 评审质疑与证据回应

如果评审只有时间看一页，就看这张表：每个质疑后面跟的是「本册哪一处能证伪它」，而不是形容词。

可能的质疑	证据回应	位置
金额是不是模型编的?	需要查账的轮次里，`ReplyGate` 拦住任何未查账先出现的 ¥/元/数字；数字只能来自本轮 `query_spend`	§3.5、`ReplyGate.java:11-13`
会不会偷偷写库?	模型只能提 `propose_add_bill`；写入发生在用户点确认之后，且写完必须回读比对	§4.2、`AssistantController.java:194-206`
没 Key 能不能用?	四个快捷句与旧意图走本地 `ReplyGenerator`；助手路径明确失败为 `ERROR`，不假装思考	§4.5、`AssistantController.java:138-141`
文档里的数字可信吗?	全部来自命令：测试 XML、`Get-Item`、`Get-FileHash`、`git rev-parse`；没有估算值	§1.3、§6.2
AI 生成的代码怎么保证质量?	51 个单测；协议文档与代码、测试三点对齐；单一实现不另立简化版	§2.5、§6.4
为什么用关键词而不是模型分类?	零延迟、可解释、可测试；漏判同义词的代价写在明处	§7.3
页数会不会超?	生成脚本以 24–30 为硬门，越界非零退出；本册页数由 pypdf 实测	§5.4

自证的限制也写在这里：以上都是代码与命令层面的证据，**不包含**真机端到端重放与真实模型质量评估（那两项在 §7.2 标为未实测），评审若需要，应按 §5.2 自行跑一遍。